

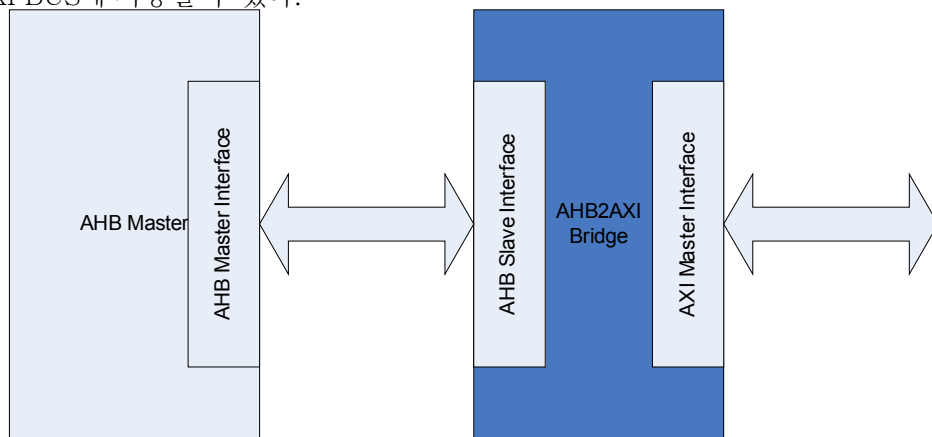
Title	AHB2AXI Bridge
Project	None
Description	AHB2AXI Bridge의 설계와 동작
Author	holelee
Date	15/May/2005

(*) AHB2AXI Bridge

AHB2AXI Bridge는 기존의 AHB Master IP를 AXI BUS에 붙일 수 있도록 해 주는 Bridge이다.(참고로, AXI Bus에 AHB Slave IP를 붙일 수 있게 해 주는 Bridge는 AXI2AHB Bridge로 별도로 설계될 예정이다.) AHB2AXI Bridge는 AHB BUS에서는 slave로 동작하며, AXI BUS에서는 master로 동작하게 된다. 큰 쓰임새는 다음과 같은 두가지 경우를 들 수 있다.

- AHB Master IP를 직접 붙이는 경우

AHB Master IP를 직접 AXI BUS로 연결하는 방법이다. ARM9과 같은 CPU를 이런 방법을 통해서 AXI BUS에 사용할 수 있다.



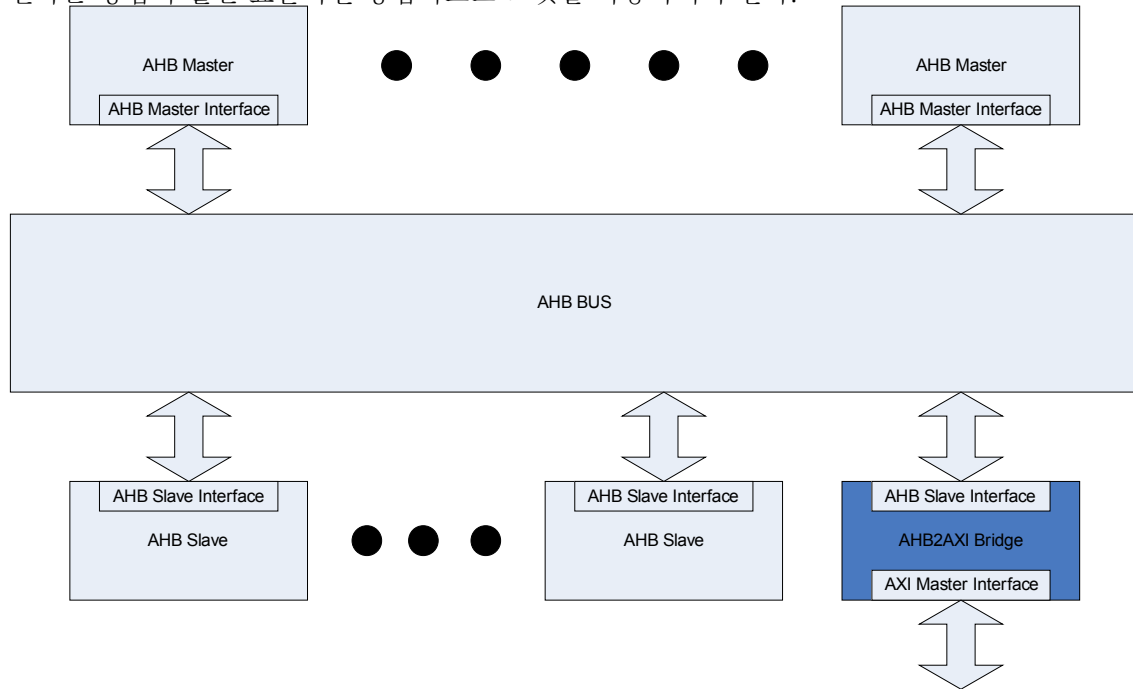
- AHB BUS를 붙이는 경우

아래 경우는 AHB BUS의 한 slave로 AHB2AXI Bridge가 사용되는 경우이다. 이 경우에 여러 개의 Master가 AHB BUS를 통해서 AXI에 접근할 수 있다. 기존에 설계된 AHB BUS platform이 있다면 그것을 크게 변형하지 않고 단순히 Slave를 하나를 추가함으로써 AXI Bus와 Interface를 수행할 수 있게 된다. 또한 기존에 설계된 AHB Master 중에서 높은 Bandwidth가 필요하지 않은 것들은 AXI에 직접 붙이기 보다 아래 그림처럼 AHB BUS로 묶어서 사용하게 되면 AXI port의 갯수가 줄어들므로 AXI BUS 속도를 좀 더 높일 수 있는 장점이 있다.

이 경우에 반드시 알아야 할 점은 AXI BUS에서 AHB BUS에 물려 있는 slave로의 접근은 불가능하다는 점이다. AXI BUS를 통해서 AHB slave에 접근하려고 한다면 위에서 언급한대로 별도로 설계할 예정인 AXI2AHB Bridge를 사용하여야 한다.

AHB2AXI Bridge는 AHB Slave로 SPLIT/RETRY 등을 사용하지 않는다. 만약 사용자가 SPLIT/RETRY capable한 AHB2AXI Bridge를 사용하여 AHB BUS의 효율성을 높이길 원한다면, AHB BUS를 Bridge로 연결하는 방법보다는 위에서 살펴본 AHB Master를 직접 AXI에 연

결하는 방법이 훨씬 효율적인 방법이므로 그것을 사용하여야 한다.



두 가지 쓰임새에 따라서 AHB2AXI Bridge의 동작이 조금 다르다. AHB BUS의 Slave로 붙는 경우 AHB2AXI Bridge는 Bus arbitration 같은 것은 신경쓸 필요 없이 단순히 Slave interface signal만 신경쓰면 되지만 AHB Master를 직접 붙일 경우에는 Bus arbitration에 관련된 처리도 수행해 주어야 하기 때문이다. 이렇게 다른 동작을 하는 두 Bridge를 별도로 만들 필요는 없고 우선 AHB BUS의 slave로 동작하는 Bridge를 만든 다음, Bus arbitration과 HSEL generation에 관련된 logic을 추가하여 AHB Master에 직접 연결되는 Bridge로 만들면 될 것으로 보인다.

(*) AHB2AXI Bridge의 동작

기본적으로 AHB Bus의 clock과 data width는 AXI BUS와 동일하다는 가정하에 설계를 진행한다. clock과 data width가 다른 경우에 대해서는 AXI쪽 interface에 clock과 data width를 바꿀 수 있는 bridge를 붙여서 해결하도록 한다. AHB BUS에 연결되는 Bridge의 경우 Split/Retry를 지원하도록 할 수도 있는데, 이런 것은 고려하지 않기로 한다. Split/Retry를 통해서 더 높은 성능이 요구되는 BUS의 경우에는 AHB Master에 연결되는 Bridge를 사용하는 것이 훨씬 효율적이기 때문에 사용자는 그러한 선택을 하여야 한다.

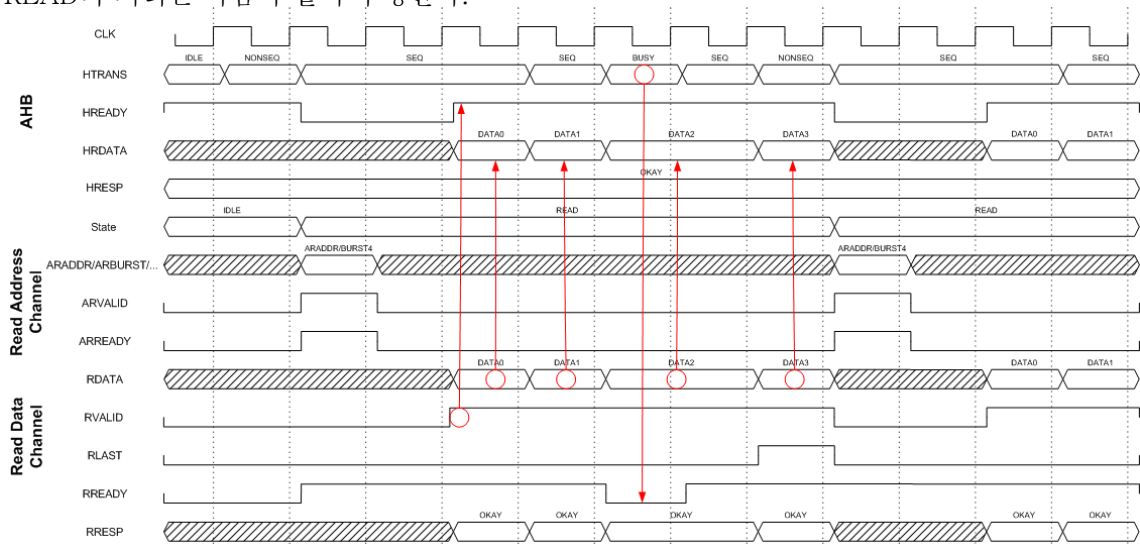
ARM에서 제공하는 AHB2AXI Bridge를 살펴보면 약 1000 gate 급으로 설계되어 있다. 이것은 곧 Bridge 내부에 FIFO(혹은 QUE)와 같은 device를 사용하고 있지 않다는 것을 의미한다. 설계할 AHB2AXI Bridge도 ARM에서 제공하는 것과 마찬가지로 FIFO와 같은 장치를 사용하지 않기로 한다. FIFO가 필요한 경우는 AHB clock과 AXI clock이 다르거나 data width가 다른

경우인데, 이런 경우에는 그 FIFO는 clock과 data width를 바꾸는 bridge 내부에 존재하게 된다.

또한 AHB2AXI Bridge의 data 전달은 combinational path를 사용하기로 한다. Timing issue가 발생하는 경우에는 AXI Register Slice 등으로 timing을 끊어서 사용하기로 한다.

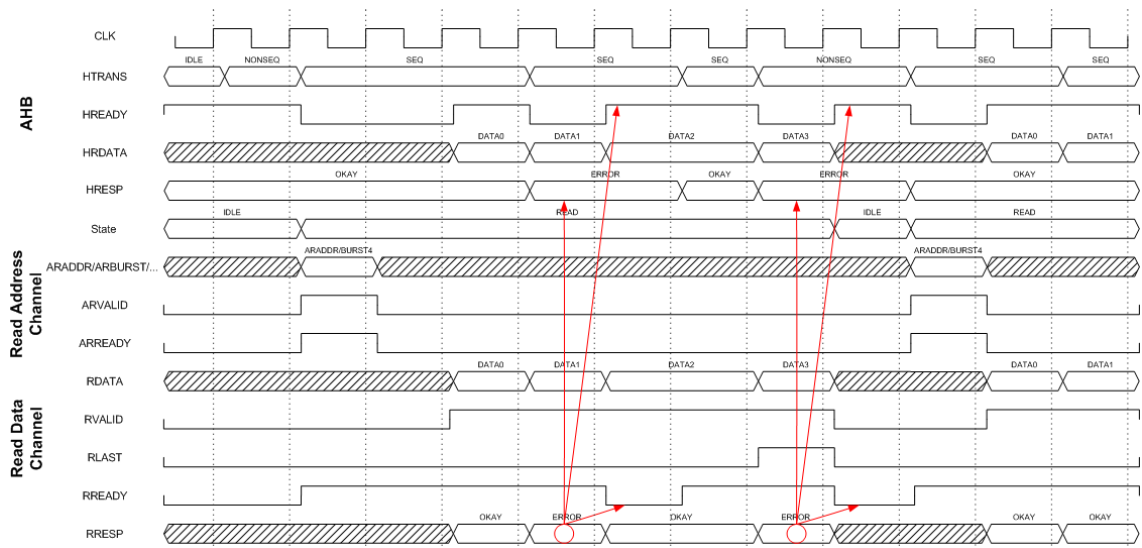
(*) READ의 처리

READ의 처리는 다음과 같이 수행한다.



AHB에서 READ request가 들어오면 우선 READY를 low로 하여 대기 시킨다. AXI쪽에 해당 read request를 보내게 된다. AXI 쪽의 RVALID/RRESP/RDATA 신호가 AHB의 HREADY/HRESP/HRDATA로 forwarding된다. RREADY 신호는 AHB의 HTRANS가 BUSY 이면 Low가 되고 아니면 High로 유지되게 된다. RLAST가 1이면 다음 request를 처리할 수 있도록 한다.

한가지 생각해 두어야 하는 문제는 AXI쪽의 RRESP가 OKAY가 아닌 경우에는 이것을 AHB쪽에 2 cycle error response로 전달해야 한다는 점이다. 아래 그림과 같이 RREADY가 1 cycle 떨어져서 다음 data를 1 cycle delay시켜 받아야 한다. IDLE state로 transition이 된 상태에도 Error Response는 전달되어야 하므로 신경써 주어야 한다.

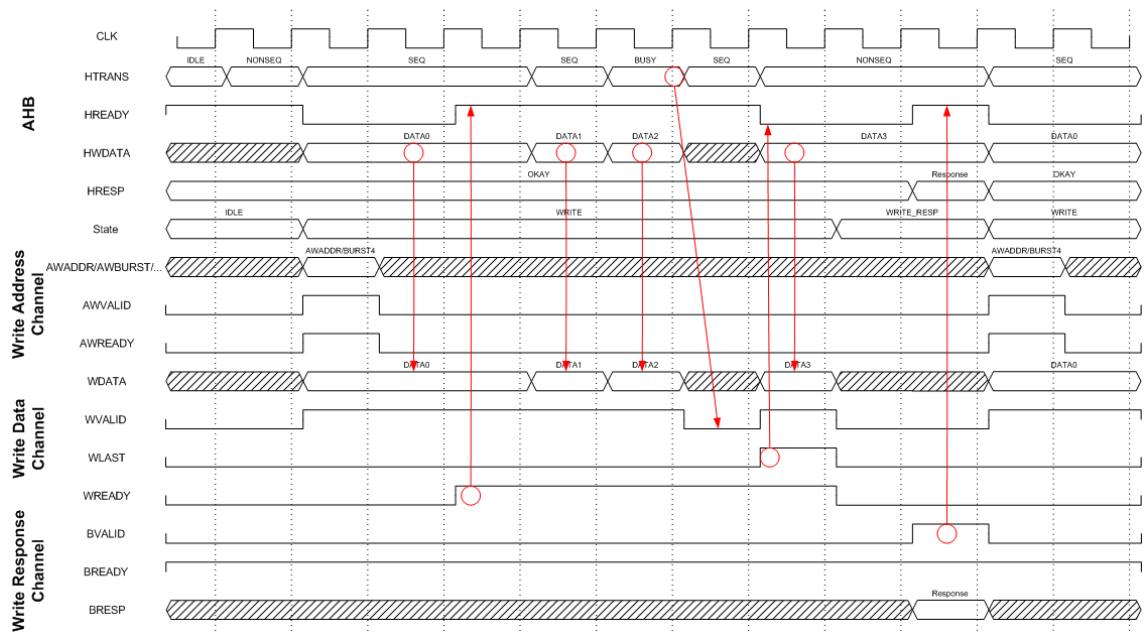


또한, AHB Master는 error response를 받았을 때 나머지 transfer를 계속 진행할 수도 있고, 그냥 중단할 수도 있다. 보통의 AHB Master의 경우에는 transfer를 계속 진행하겠지만, 중단하는 경우에 대해서도 대비를 해야 하는데, 우선은 계속 진행한다고 가정하고 설계한다.

(*) WRITE의 처리

WRITE의 처리도 READ의 처리와 비슷하다. WVALID 신호는 HTRANS를 latch한 값으로 주면 된다. HTRANS가 NONSEQ나 SEQ인 경우 WVALID가 high가 되고 BUSY인 경우에는 WVALID를 low로 주게 된다. HREADY는 WREADY 신호를 직접 forwarding한다.

다만 문제가 되는 것은 AXI는 Write Response를 모든 data가 전송된 다음에 받도록 되어 있는데, AHB는 그렇지 않다는 점이다. AHB쪽에는 Burst write 중에는 계속 OKAY response를 전달하다가 맨 마지막 data를 write할 시점에서 AXI의 response를 전달하도록 한다. 그렇게 하기 위해서는 맨 마지막 HWDATA에 대해서는 HREADY를 low로 하여 상당기간 delay를 주어야 한다.



이와 같은 경우에 대해서 문제가 되는 경우가 한가지 발생할 수 있는데, HTRANS에 BUSY가 떠서 WVALID가 low로 떨어진 경우에 AXI slave쪽에서 WREADY를 low로 내리고 WVALID가 high가 될 때까지 대기하게 되면 문제가 된다. WREADY가 low이므로 HREADY가 계속 low가 되므로 AHB쪽에서 transfer가 제대로 일어나지 못하게 되는데 이 부분을 처리하기 위해서는 WVALID가 low이고 HTRANS가 BUSY가 아니면 HREADY를 high로 하여 WVALID가 high가 되도록 만들어야 한다.

일반적인 경우에는 Write Response를 굳이 볼 필요가 없다. CPU의 경우에도 Write에서 error가 발생했다고 해도 결국 다음번 read에서 error를 받으면 큰 문제가 없이 동작을 할 것이다. 이런 경우에는 굳이 맨 마지막 data write를 delay 시킬 필요는 없고 단순히 모든 response를 OKAY로 하고 Write를 빨리 처리하는 방법을 생각해 볼 수도 있다. 이런 경우에는 Write Response가 오지 않았을 경우에 Read request를 처리하지 말아야 하는데, 이것을 고려해야 한다.

(*) Locked access의 처리

AHB2AXI Bridge의 경우 master가 locked access를 사용하고 있는지를 HMASTLOCK 신호로 알 수 있다. HMASTLOCK은 원래의 AHB BUS에서는 arbiter가 만들어서 slave로 전달해주는 신호인데, 이 signal의 사용에 대해서는 완벽한 설명이 존재하지 않는다. 예를 들어 ARM9 CPU가 swap 명령을 수행하였을 때, 맨 처음 read는 반드시 locked access인데, 그 다음 수행하는 write의 경우 locked access로 처리하는지 아니면 일반 unlocked access로 수행하는지 확실하지 않다. 만약 read/write 모두를 locked access를 수행한다면, 문제가 발생할 수 있다. AXI의 locked access는 어떤 master가 locked access를 시작하면 그 끝은 반드시 normal

access로 끝나도록 되어있다. Locked access read/write를 수행한 다음 CPU가 새로운 transaction을 수행하지 않고 대기한다면 AXI bus는 계속 locked 상태로 남아 있을 수 밖에 없다. 따라서 이부분에 대한 확인과 함께 어떻게 대처를 해야 하는 지 생각해 볼 필요가 있다.

(*) INCR burst의 처리

AHB에서 지원하는 undefined length INCR burst는 AXI에는 존재하지 않는다. 따라서 INCR burst의 경우에는 single transfer로 분할하여 전송하도록 한다.

(*)WSTRB는 어떻게 생성해야 하나?

AHB에는 ByteEnable에 관련된 내용은 없다. 따라서 별도의 logic이WSTRB를 생성해 주어야 하는데 비교적 간단하게 만들 수 있다. HADDR, HSIZE을 latch한 후에 그 값에 따라서WSTRB 신호를 combinational하게 출력해 주면 된다.

(*) 설계된 AHB2AXI Bridge

설계가 끝난 AHB2AXI Bridge는 Test용 AHB-lite Master와 AXI Internal SRAM Controller를 연결하여 random read/write test를 수행하여 검증하였다.(참고로 아직까지 Locked access에 대해서는 검증을 진행하지 않았다.)

AHB2AXI Bridge는 기본적으로 AHB Slave Interface를 가지고 Locked access를 위해 HMASTLOCK이라고 하는 입력 신호를 받는다.

- AHB Master와의 직접 연결 방법

AHB Master가 AHB-lite Master라면 별 문제 없이 그냥 연결할 수 있다. Master의 HLOCK신호를 Bridge의 HMASTLOCK에 연결하고, 그 다음 Bridge의 HREADY_IN에 자신의 출력인 HREADY_OUT을 연결하면 된다. Bridge의 입력인 HSEL에는 1'b1을 연결하여 주면 된다.

AHB Master가 일반적인 AHB BUS Master라면 arbitration을 위해서 HGRANT 신호를 1'b1로 연결하여 주고, HREQ 신호는 open 상태로 둔다. Master의 HLOCK 신호를 HMASTLOCK으로 timing을 변경하여 연결할 필요가 있다. HSEL과 HREADY_IN/OUT의 연결은 AHB-lite Master를 연결하는 경우와 같다.

- AHB BUS에 연결 방법

AHB BUS에의 연결은 그냥 Bridge의 모든 AHB interface를 보통의 AHB Slave 연결하듯이 연결하면 된다.